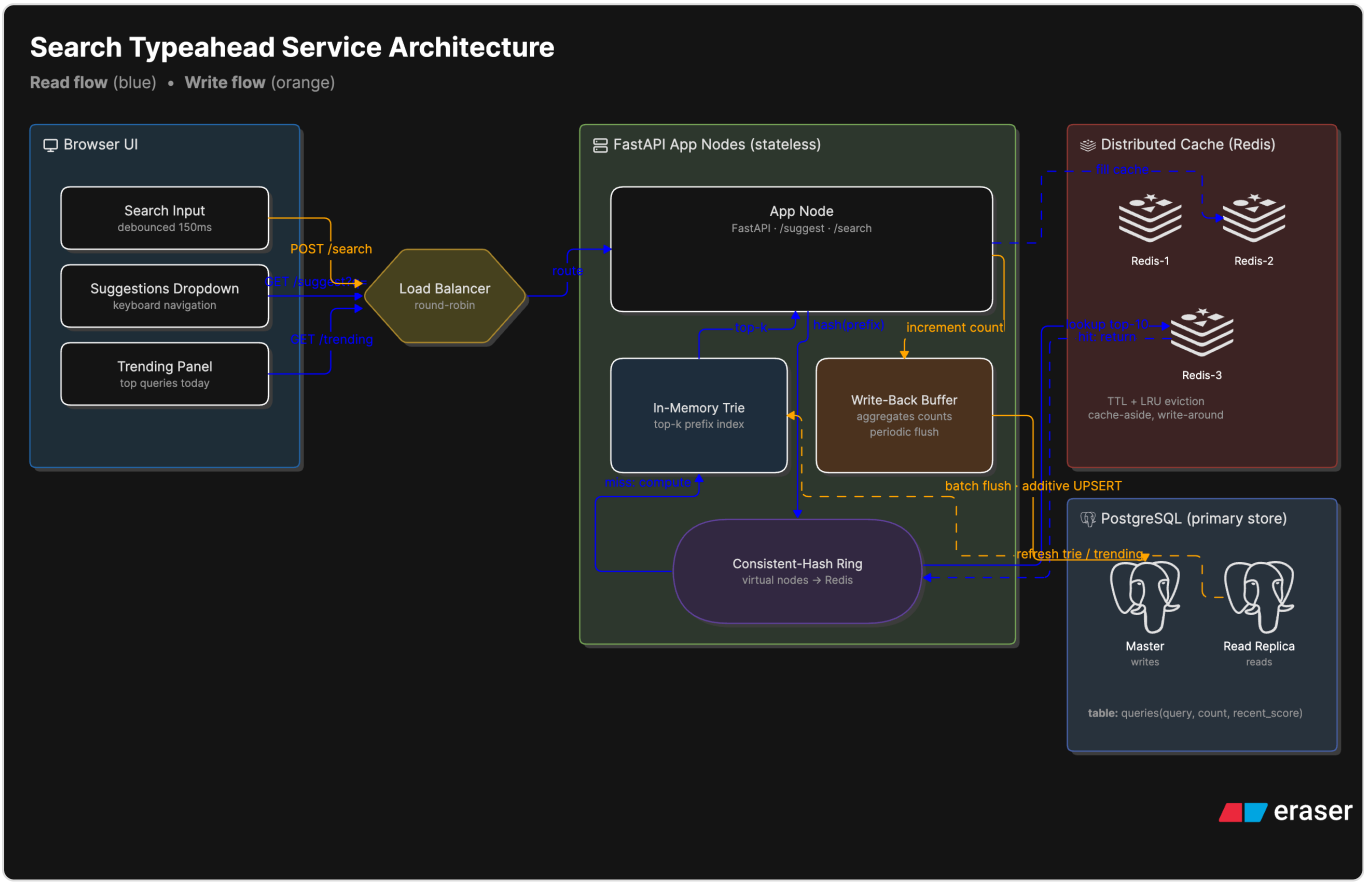
Search Typeahead System

Project Report — low-latency search autocomplete with a distributed cache, consistent hashing, write-back batching, and recency-aware trending.

Stack: Node.js (Express) · PostgreSQL · 3 Redis nodes · vanilla-JS UI I Dataset: AOL 2006 query log (top 1,000,000 queries)

---

1. Architecture



The system has two opposing workloads on one dataset: **reads** (suggestions, fired on every keystroke) and **writes** (count updates on every submit). Reads dominate ~5–10:1. The architecture optimizes the read path while keeping writes cheap, and accepts bounded staleness because suggestion data is approximate popularity.

Components

- **Browser UI** — debounced input, suggestion dropdown with keyboard navigation, a trending panel, and a count-vs-hybrid ranking toggle.
- **Load balancer** — round-robin in front of stateless app nodes (any node serves any request).
- **App node (Node.js/Express)** — holds an in-memory **trie** (top-k prefix index), a **write-back buffer**, and a **consistent-hash ring** that routes a prefix to a cache node.
- **Distributed cache** — 3 Redis nodes, each with TTL expiry and LRU eviction; the global, shared cache layer.
- **PostgreSQL** — durable source of truth (`query, count, recent_score, last_searched`); master + read replica.

Read path — `GET /suggest`

Normalize the prefix → the consistent-hash ring picks the owning Redis node → on a **hit** return the cached top-10; on a **miss**, compute the top-10 from the in-memory trie, store it in that node with a (jittered) TTL, and return it. The DB is never on the suggestion path — a miss is served by the trie.

Write path — `POST /search`

Return `{"message":"Searched"}` immediately (a synchronous *acknowledgement*), then increment an in-memory buffer. The buffer aggregates duplicate queries and flushes on size or time into one additive `UPSERT`. The cached entries for affected prefixes self-heal via TTL expiry (write-around). So the response is synchronous, but the count update — and its visibility in suggestions — is asynchronous.

2. Dataset — source & loading

Source: AOL 2006 search query log (~36M real user queries). Counts are **derived by aggregation** (`COUNT(*)` per normalized query), exactly the assignment's "derive counts if absent". Query popularity is naturally Zipf/Pareto, which is what makes caching effective. Mirror: Internet Archive (no login). Privacy note: only query text + derived counts are used; all user IDs are dropped.

Load the AOL dataset:

```
curl -L -o files/aol.zip "https://archive.org/download/AOL_search_data_leak_2006/AOL_search_data_leak_2006
unzip -o -j files/aol.zip "AOL-user-ct-collection/*.txt.gz" -d files/aol_data
node --max-old-space-size=4096 scripts/loadDataset.js --dir files/aol_data --min-count 2 --out files/aol_a
node scripts/loadDataset.js --agg-file files/aol_agg.tsv --top 1000000 --min-count 3
```

35.4M rows aggregate to 4.1M distinct queries; we load the **top 1,000,000 by count** (the full set makes the in-memory trie unnecessarily large). The loader auto-detects the delimiter and `Query` column, lowercases/trims, drops blanks and `-`, and bulk-loads into Postgres. A no-download option, `node scripts/loadDataset.js --synthetic 120000`, generates Zipf-distributed queries and still exceeds the 100k minimum.

3. API documentation

Method	Endpoint	Behavior
GET	`/suggest?q=<prefix>&mode=count|hybrid`	Up to 10 prefix matches. `count` = all-time popularity; `hybrid` = recency-aware. Returns `source` (cache/trie) and owning `node`.
POST	`/search` `{"query":"..."}`	Returns `{"message":"Searched"}` immediately; buffers the count (write-back).
GET	`/cache/debug?prefix=<p>`	Which cache node owns the prefix, its hash, ring position, and HIT/MISS.
GET	`/cache/ring?sample=N`	Key distribution of N sample prefixes across the nodes (balance evidence).
GET	`/trending?n=10`	Global trending by decayed recent score.
GET	`/metrics`	Cache hit rate, DB read/write counts, write-reduction factor, p50/p95/p99.

```
curl 'http://localhost:8000/suggest?q=goog&mode=hybrid'
curl -X POST 'http://localhost:8000/search' -H 'Content-Type: application/json' -d '{"query":"iphone 15"}'
curl 'http://localhost:8000/cache/debug?prefix=ip'
curl 'http://localhost:8000/trending?n=10'
```

4. Design choices & trade-offs

Caching — why, and what kind

- **Need:** ~7k peak read QPS would saturate Postgres if every keystroke ran a prefix scan + sort. Query traffic is Zipf, so a cache over the hot set gives a high hit rate and keeps reads off the DB.
- **Global vs local → global:** one shared copy and one place to invalidate; per-node local caches duplicate data and need cross-node invalidation to stay coherent.
- **Single vs distributed → distributed (3 Redis):** a single node would handle this load fine; we distribute for **fault tolerance** (one node down loses 1/3 of the cache, not all → no full DB stampede) and headroom — not throughput.

Routing — consistent hashing

Cache nodes hold different keys, so routing must be deterministic per key (round-robin loses *where* a key lives). `hash % N` is deterministic but remaps almost all keys when N changes; a hash ring with virtual nodes remaps only ~K/N keys on add/remove. The app tier itself is stateless, so it uses a plain round-robin load balancer.

Eviction & invalidation

- **Eviction → LRU:** hot prefixes are re-hit constantly so LRU keeps them and evicts the rare long tail. LFU fits stable Zipf slightly better but needs an aging factor; not worth it, especially since short TTLs mean eviction rarely fires.
- **Invalidation → write-around + short TTL:** TTL (≈45s suggestions, ≈8s trending, jittered) bounds staleness with zero tracking. We avoid write-through because the cache value is a computed top-10 — recomputing it on every write is wasted work. Targeted invalidation was skipped as unnecessary complexity.

Writes — write-back batching

`POST /search` increments an in-memory map and returns immediately. The buffer aggregates duplicates and flushes on size **or** interval into one additive `UPSERT` (`count = count + EXCLUDED.count`, so concurrent flushes add instead of clobber). **Trade-off:** a crash loses at most one un-flushed window — acceptable for approximate, self-healing counts; durability would require a write-ahead log or a durable queue (Kafka/Redis Streams).

Store — PostgreSQL (not NoSQL/LSM)

The workload looks write-heavy, which usually argues for a write-optimized LSM store. But batching already cut DB writes ~60×, removing that pressure — so we use Postgres: a B-tree index serves `LIKE 'pre%'` range scans, ACID protects the count truth, and read replicas scale reads. An LSM store (Cassandra) would only be warranted with un-batchable, massive writes or a need for sharding + quorum. (The batch buffer is itself a memtable-like idea at the app layer; Redis, the cache, is the one NoSQL store we use.)

Consistency — eventual (PA/EL)

Suggestion data tolerates staleness but not latency or downtime. So during a partition we serve stale rather than error (AP), and in normal operation we serve from cache rather than re-check the DB (EL). This single posture is the root of the caching, TTL, and write-back choices.

Serving & ranking

- **Trie with lazy top-k:** O(prefix length) lookups. We precompute candidate pools only for short prefixes (≤3 chars — few but broad/hot); longer prefixes are computed on demand. We do not precompute every prefix (tens of millions of nodes).
- **Ranking:** `count` = all-time; `hybrid` = `w_pop·log(count) + w_rec·recent_score`, where `recent_score` increments per search and decays exponentially (1h half-life), so a short-lived spike fades instead of ranking forever. The same trie serves both modes.

Topic	Decision	Rejected alternative
Cache locality	Global	Local (duplication + coherence)
Cache topology	Distributed (3)	Single (SPOF / stampede)
Routing	Consistent hashing + vnodes	Round-robin (no locality), `%N` (mass remap)
Eviction	LRU	LFU (needs aging)
Invalidation	Write-around + short TTL	Write-through (costly), targeted (complex)
Writes	Write-back batching	Sync per-write (DB-bound)
Store	PostgreSQL	NoSQL/LSM (unneeded after batching)
Consistency	Eventual / PA-EL	Strong (latency cost)
Serving	Trie + lazy top-k	Full precompute (waste), DB-only (slow)
Ranking	Count + decayed hybrid	Raw recent counter (over-ranks spikes)

5. Performance report

1 Node.js app node, 1 Postgres, 3 Redis nodes (Docker). Dataset: AOL 2006, top 1,000,000 queries. Generated with `node scripts/benchmark.js --reads 8000 --writes 20000`.

Read latency & cache hit rate

Metric	Value
Server p95 (`/suggest`)	~0.5–0.75 ms
Client p95 / p99	~1.25 ms / ~3.9 ms
Cache hit rate	~83% (rises to 90%+ with more skewed load / longer TTL)
DB reads on suggestion path	0 (a miss is served by the in-memory trie)

Write reduction (batching)

Searches received	Rows written	Flush transactions
20,000	~4,700	10

≈ 4.3× fewer rows (duplicate aggregation) and ~2,000× fewer transactions (batching).

Consistent-hashing distribution

5,000 sample prefixes across 3 nodes (150 virtual nodes each): **30.9% / 34.6% / 34.5%** — within ~±4% of even. Adding/removing a node remaps only ~1/N of keys. Per-prefix routing is inspectable via `GET /cache/debug?prefix=<p>`.